

BAGIAN 4 DARI 8

Sesi 2 — Skema Bintang

Sesi ini mengubah satu tabel lebar `penjualan_bersih` menjadi satu tabel fakta dengan empat dimensi di sekelilingnya — bentuk bintang yang jadi bahasa bersama semua pembuat gudang data.

Grain ditetapkan lebih dulu, sebelum satu kolom pun ditulis

Satu baris di `fact_penjualan` mewakili **penjualan satu produk pada satu nota**.

Satu kalimat itu menentukan segalanya: ukuran mana yang boleh dijumlahkan, dimensi mana yang perlu ada, dan query mana yang mustahil dijawab. Nota berisi tiga produk menghasilkan tiga baris. Biasakan menulis kalimat grain di komentar paling atas berkas SQL-mu, sebelum `CREATE TABLE`.

Dimensi tanggal dibuat sendiri, satu tahun penuh

```
INSERT INTO dim_tanggal
WITH RECURSIVE deret AS (
  SELECT DATE('2026-01-01') AS tgl
  UNION ALL
  SELECT tgl + INTERVAL 1 DAY FROM deret WHERE tgl < '2026-12-31'
)
SELECT tgl,
       YEAR(tgl), MONTH(tgl), MONTHNAME(tgl),
       DAY(tgl), DAYNAME(tgl), WEEK(tgl, 1), QUARTER(tgl),
       CASE WHEN DAYOFWEEK(tgl) IN (1, 7) THEN 'Ya' ELSE 'Tidak' END
FROM deret;
```

Hasilnya 365 baris, padahal transaksi hanya ada di 90 hari. Itu disengaja. Dimensi tanggal dibuat untuk seluruh kalender, bukan hanya untuk hari yang ada penjualannya — karena pertanyaan “hari apa saja toko tidak menjual apa pun?” baru bisa dijawab kalau hari-hari kosong itu punya baris di dimensi.

Pemeriksaan nomor 5 di sesi ini membuktikannya: `LEFT JOIN` dari `dim_tanggal` ke fakta menemukan **275 hari tanpa penjualan**. Tanpa dimensi tanggal yang lengkap, angka itu tidak mungkin keluar dari query apa pun.

Dimensi produk juga menambah kolom yang tidak ada di sumber mana pun:

```
CASE WHEN harga < 10000 THEN 'Murah'
      WHEN harga < 25000 THEN 'Sedang'
      ELSE 'Mahal' END
```

Ini contoh paling kecil dari gagasan besar: dimensi bukan salinan tabel sumber, dimensi adalah tempat menaruh cara bisnis memandang sesuatu.

Dua rancangan yang menyelamatkan laporan

```
-- Baris id 0 untuk pembeli yang tidak menyebutkan identitas
INSERT INTO dim_pelanggan VALUES (0, 'Tanpa Pelanggan', '-', NULL);
```

```
SELECT tanggal, nama_produk, nama_cabang,
       COALESCE(id_pelanggan, 0), -- bukan dibuang, diarahkan
```

```
no_nota, jumlah, harga_satuan, total
FROM penjualan_bersih;
```

Ada 167 baris transaksi tanpa identitas pembeli. Kalau `NULL` dibiarkan, `INNER JOIN` ke dimensi pelanggan akan menghapus 167 baris itu dan omzet laporan langsung meleset. Baris “Tanpa Pelanggan” membuat semua transaksi tetap ikut terhitung sambil tetap bisa dipisahkan.

Kolom `no_nota` tidak punya tabel dimensi sendiri — ia duduk di tabel fakta sebagai **degenerate dimension**. Tidak ada atribut lain yang bisa digantung padanya, jadi membuat `dim_nota` hanya akan jadi tabel satu kolom yang tidak berguna.

Titik periksa

Pemeriksaan	Nilai benar
<code>dim_tanggal</code> / <code>dim_produk</code> / <code>dim_cabang</code> / <code>dim_pelanggan</code>	365 / 8 / 3 / 41
<code>fact_penjualan</code>	560 baris, omzet 20.784.000
Nota + produk kembar	0 baris
Omzet per kategori (Kue / Roti / Donat)	13.801.000 / 5.166.000 / 1.817.000
Omzet per hari: akhir pekan vs hari kerja	301.808 vs 202.141
Hari tanpa penjualan	275

Dimensi pelanggan berisi 41 baris, bukan 40: empat puluh pelanggan ditambah satu baris “Tanpa Pelanggan”. Periksa angka ini sebelum lanjut ke tabel fakta.

Angka omzet 20.784.000 harus **sama persis** dengan hasil Sesi 1. Memindahkan data ke skema bintang tidak boleh menambah atau mengurangi satu rupiah pun.

Jebakan ukuran non-additive

```
SELECT SUM(harga_satuan) AS jangan_pernah_di_sum,
       ROUND(AVG(harga_satuan)) AS rata_rata_tanpa_bobot,
       ROUND(SUM(total) / SUM(jumlah)) AS harga_rata_rata_benar
FROM fact_penjualan;
```

Cara menghitung	Hasil	Benar?
<code>SUM(harga_satuan)</code>	8.870.000	tidak bermakna apa pun
<code>AVG(harga_satuan)</code>	15.839	salah, tiap baris dihitung sama bobotnya
<code>SUM(total) / SUM(jumlah)</code>	16.225	benar

Tiga angka dari satu tabel yang sama, dan hanya satu yang boleh masuk laporan. `jumlah` dan `total` bersifat additive — boleh dijumlahkan ke arah dimensi mana pun. `harga_satuan` tidak: menjumlahkan harga satuan tidak menghasilkan apa-apa, dan merata-ratakannya tanpa bobot memberi harga roti murah yang terjual banyak pengaruh yang sama dengan lapis legit yang terjual sedikit.

Selisih 15.839 melawan 16.225 memang kecil. Justru itu yang berbahaya: angka salah yang kelihatan masuk akal lolos dari pemeriksaan jauh lebih sering daripada angka salah yang jelas ngawur.

Modul Praktikum Data Warehouse · Paket Inti · Bagian 4 dari 8 · Seluruh kode dalam bagian ini sudah diuji jalan